THIAGARAJAR COLLEGE (AUTONOMOUS), MADURAI - 625009
Department of Computer Science | B.Sc. Computer Science (2026)

SECURE CHIT: DIGITAL CHIT FUND MANAGEMENT SYSTEM

COMPREHENSIVE PROJECT VIVA-VOCE QUESTION & ANSWER GUIDE

Students: R. NARESHKUMAR (24UCS08) APR. JEGATHEESH VARSHAN (24UCS06)	**Project Guide:** Mrs. SM. VALLI B.E., M.C.A., M.Phil **HOD,** Dept. of Computer Science

1. System Architecture & Tech Stack Summary

Component	Technology Used	Key Role in SecureChit
Frontend UI	React 18 + Tailwind CSS	Component-based reactive UI, real-time bid monitoring dashboards, responsive styling without CSS overhead.
Digital Signature	HTML5 Canvas Pad	Interactive mouse/touchpad for drawing signatures onto legal Promissory Notes (PRN-YYYY-XXXX).
Backend API	Python FastAPI	High-performance asynchronous REST endpoints, Swagger UI documentation, background task workers.
Winner Allocation	AI Priority Evaluation Engine	Analyzes emergency score parameters to fairly prioritize winning bids rather than blindly lowest bids.
Database	PostgreSQL	Strict ACID compliance, relational integrity, immutable financial ledgers and transaction audit trail.
Payment Gateway	Razorpay API	Instant online monthly installment payments with automated digital receipt generation.
Notifications	Fast2SMS & SMTP Email	Automated pre-due installment reminders and win announcements to guarantee zero default rates.
Security & Auth	JWT + bcrypt + RBAC	Role-Based Access Control (Admin / Member), salted password hashing, stateless Bearer token authorization.

2. Technical & Domain Viva-Voce Questions & Answers

Q1: What is the primary objective of your project 'SecureChit'?

Answer: The primary objective of SecureChit is to digitize and automate traditional chit fund operations. It replaces manual ledgers, paper promissory notes, and informal bidding with a transparent, cloud-based platform featuring live priority-based bidding, instant digital receipts, legal promissory note e-signatures, and automated dividend distribution.

Q2: How does a traditional Chit Fund operate, and how does SecureChit modernize it?

Answer: In a traditional chit fund (ROSCA), all members contribute a fixed sum monthly to create a pool. In ordinary schemes, members bid down the pot (reverse auction). However, SecureChit replaces this with an **AI-powered priority-based bidding model** that evaluates urgent emergency priorities to award the fund ethically. It accurately computes the 5% foreman commission and distributes remaining dividend discounts among all non-winning members while keeping an immutable digital audit ledger.

Q3: Why did your project use Priority-Based Bidding instead of a simple Reverse Auction?

Answer: In traditional reverse auctions, speculative bidders or financially aggressive participants bid down excessively, depriving genuinely needy members who have real medical, educational, or business emergencies. SecureChit's Priority Bidding Engine evaluates verified priority scores and genuine emergency parameters so that chit allocations are equitable, fair, and transparent without bid manipulation or collusion.

Q4: What are the primary functional modules implemented in SecureChit?

Answer: The system consists of 7 tightly integrated modules:
1. **Member Onboarding & KYC:** Digital member registration and verification.
2. **Role-Based Access Control (RBAC):** Dedicated dashboards for Admin (Foreman) and Members.
3. **Priority-Based Bidding Engine:** Bid submission and emergency assessment.
4. **AI Smart Winner Allocation Engine:** Fair multi-factor evaluation of winner priorities.
5. **Pronote Management (HTML5 Canvas):** Legal Promissory Note creation with touch signature capture.
6. **Payment Gateway & Digital Receipts:** Razorpay integration for online installments.
7. **Automated Communication Engine:** Fast2SMS & SMTP email reminders.

Q5: Why did you choose Python over Java for the backend server?

Answer: We selected Python over Java for three compelling engineering reasons:
1. Native AI/Algorithmic Ecosystem: Python is the global standard for AI, priority weighting algorithms, and data analysis. Integrating our Smart Winner Allocation Engine directly within Python is seamless and lightweight.
2. Rapid Development & Minimal Boilerplate: Java frameworks (like Spring Boot) require excessive boilerplate code (DTOs, DAOs, XML/annotations). Python's concise syntax drastically accelerated our sprint delivery.
3. High Asynchronous Concurrency with FastAPI: Using Python's modern async/await paradigm via Starlette and Uvicorn, FastAPI provides microsecond throughput comparable to Node.js and Go, handling concurrent bidding without the heavy JVM memory overhead.

Q6: Why did you choose FastAPI over other Python frameworks like Django or Flask?

Answer: While Django is monolithic and Flask is synchronous by default, FastAPI was built from the ground up for modern asynchronous microservices. It features automatic schema validation using Pydantic, auto-generates interactive OpenAPI/Swagger documentation, and natively supports async/await, making it ideal for real-time bid processing.

Q7: Why was PostgreSQL chosen over NoSQL databases like MongoDB?

Answer: Chit funds are financial accounting systems where balances, ledger audits, and double-entry integrity are paramount. PostgreSQL guarantees strict **ACID (Atomicity, Consistency, Isolation, Durability)** properties and relational foreign key constraints. In contrast, document databases like MongoDB can lead to eventual consistency anomalies and ledger discrepancies during simultaneous bid/payment transactions.

Q8: How does the Promissory Note (Pronote) digital signature feature work?

Answer: Upon winner declaration, the system auto-generates a unique legal Promissory Note record (PRN-YYYY-XXXX) marked with status 'Pending Signature'. On the member dashboard, an interactive HTML5 Canvas element allows the winner to draw their touch or stylus signature. The signature vector is converted to Base64 image data, securely transmitted via authenticated API, stored in PostgreSQL, and stamped onto the formal digital document, making it visible to the Admin.

Q9: How is online payment and installment tracking handled?

Answer: We integrated the **Razorpay Payment Gateway API**. When a member clicks 'Pay Installment', Razorpay handles card/UPI/net-banking processing. Upon transaction completion, SecureChit verifies the payment signature webhook, updates the member ledger immediately, recalculates outstanding dues, and generates an instant printable digital payment receipt.

Q10: How does SecureChit minimize payment defaults?

Answer: SecureChit prevents defaults proactively using a multi-channel automated communication dispatcher. Integrated with **Fast2SMS** for transactional SMS and **SMTP (Nodemailer)** for email, the backend runs scheduled cron jobs sending automated reminder alerts 3 days and 1 day prior to due dates, as well as instant win declarations and receipts.

Q11: Explain the security mechanisms implemented in the project.

Answer: Security is enforced at every layer:
• **Password Hashing:** Uses bcrypt with random cryptographic salt.
• **Stateless Authentication:** Uses JWT (JSON Web Tokens) with HMAC-SHA256 signing and expiration timestamps.
• **Role-Based Access Control:** Backend middleware blocks unauthorized endpoints (e.g., members cannot access chit creation or approval routes).
• **Data Sanitization:** Pydantic schemas prevent SQL injection and cross-site scripting (XSS).

Q12: Can you describe the Data Flow (DFD Levels) of the system?

Answer: At **Level 0 (Context Diagram)**, entities are Member, Admin, Razorpay, and SMS Gateway. Inputs include KYC, bids, and payments; outputs include digital receipts, winning alerts, and MIS reports. **Level 1** breaks down the application into User Auth, Chit Lifecycle Manager, Priority Bid Evaluator, Payment Processing, and Notification Engine. **Level 2 & 3** detail database reads/writes to tables like users, chit_groups, bids, pronotes, and payments.

Q13: What was the most significant technical challenge during development?

Answer: The biggest engineering hurdle was synchronizing the priority-based bidding evaluation in real time while maintaining transactional atomicity across database tables. Handling the HTML5 Canvas vector signature conversion, securely encrypting the Base64 payload, and locking the state so a member cannot re-sign or tamper with an issued promissory note was also a key challenge successfully solved.

Q14: What future enhancements can be added to SecureChit?

Answer: Future roadmaps include:
1. Mobile App build using React Native for Android and iOS.
2. Integration with Government DigiLocker and Aadhaar e-KYC APIs.
3. Hyperledger / Ethereum smart contracts to achieve fully decentralized, tamper-proof chit ledgers.

3. Spoken Tanglish Viva Cheat Sheet (Examiner kitta epdi pesanum)

Viva hall-la examiner kitta Tamil + English (Tanglish)-la explain panna idho standard speaking points:

1. Project Introduction (1-Minute Opening Pitch)

*"Good morning Sir / Mam. Namma project title **'Secure Chit'**. Normal-a chit fund-la manual paperwork, ledger disputes, and bid collusion irukkum. Idha resolve panna nanga oru complete digital web platform develop pannirukom. Namma project-oda main highlight enna-na: **AI-based Priority Bidding**, HTML5 Canvas moolama **Promissory Note Digital Signature**, **Razorpay Online Payment**, and automated SMS/Email alerts to prevent payment defaults."*

2. Why Python over Java? (Crucial Viva Question)

"Sir, Python choose panna 3 main reasons irukku:
***1. AI & Priority Logic:** Namma project-la AI priority allocation algorithm irukku. Python-la AI libraries integrate panradhu romba direct and easy, Java-la idhu romba complex.*
***2. Fast Development & Clean Code:** Java-la neraya boilerplate code (getters, setters, configs) ezhudhanum. Python syntax clean-a irundhadhala nanga core features-la focus panna mudinjadhu.*
***3. FastAPI Real-time Performance:** Nanga FastAPI use panrom. Idhula built-in async/await irukkaradhala live bidding-la concurrent requests-ah hang aagama high speed-la handle pannum. Java JVM memory-ah vida idhu lightweight."*

3. Priority Bidding vs Reverse Auction (Important Clarification)

*"Sir, normal chit fund-la **Reverse Auction** nadakkum (yaaru kammiya bid pandrangalo avanga winner). Aana SecureChit-la nanga **Priority-Based Bidding** implement pannirukom. Because emergency thevai irukkura members (medical, education emergencies) pathu namma AI algorithm priority calculate panni fair-a allocate pannum. Idhanala bid collusion and unfair bidding thadukkappadudhu."*

4. Promissory Note & HTML5 Canvas Signature

*"Chit win pannavangala Promissory Note sign panna solluvom. SecureChit-la win aana udane system automatic-a legal Promissory Note (PRN code) generate pannum. Winner avaroda phone or laptop screen-la **HTML5 Canvas** use panni digital sign poduvanga. Sign pannadhukku approm dhan admin portal-la amount release aagum. Paperwork complete-a zero aayirum."*

5. Database & Security (PostgreSQL, JWT, bcrypt)

*"Money transactions irukkaradhala nanga **PostgreSQL** use pannirukom, because it strictly follows ACID properties. Passwords ellam **bcrypt** algorithm vachi hash panni store aagum. User session and login-ku **JWT tokens** use panrom, so safe and secure-a irukkum."*

4. Examiner Tricky Questions & Smart Answers

Trap Q: What if two members have the exact same emergency priority score?

Smart Answer: The AI engine evaluates secondary tie-breaking parameters: past prompt payment track record (credit reliability score), number of months already participated without claiming chit, and timestamp priority of the bid submission.

Trap Q: If member loses internet connection during payment, will money be lost?

Smart Answer: No. We use **Razorpay Webhooks**. Even if the member's browser crashes or internet disconnects, Razorpay's server sends an automated backend webhook confirmation to our FastAPI server, which verifies the cryptographic signature and safely updates the database ledger.

Trap Q: Why not use MongoDB if JSON data is convenient?

Smart Answer: Chit funds deal with financial ledger balances. MongoDB has eventual consistency and lacks strict multi-table relational schema constraints. In contrast, PostgreSQL ensures strict ACID transactions, ensuring debits and credits always match without data corruption.

Trap Q: How is the 5% foreman commission handled mathematically?

Smart Answer: Total Pool = (Monthly Installment × Total Months). Foreman Commission = 5% of Total Pool. Remaining pot minus discount is allocated to winner, and the discount balance is equally credited back to all participating members as dividend deductions in their next installment.